SecureBuild: Team Workload

Roles

Person	Role	Owns
F1	Frontend Lead	Login, repo list, findings table, filters, refresh and diff UI
F2	Frontend, then Backend Helper	SBOM tab and detail drawer, then (from D5) API endpoints, the fallback lookup and tests
B1	Backend Lead	Login/JWT, database, job queue, API, worker
B2	Scanner Engineer	Clone repo, read lockfiles, ask OSV, calculate scores, pick fix versions

Parallel Work

Days	F1	F2	B1	B2
D1-D3	Login, repo list and add-repo screens (fake data)	Findings table and detail drawer (fake data)	Register/login, DB models, queue setup	Clone repo, read `package-lock.json`, call OSV
D4-D8	Filters, refresh button, scan history, diff badges	SBOM tab, then help B1 with APIs from D5	API endpoints and worker (clone, scan, save)	`poetry.lock`, scoring, fix versions, save component list
D9-D11	Connect screens to the real API, one at a time	Connect screens, fix bugs, write docs	Test on 10 repos, security fixes	Fix scanner bugs, write docs

Feature freeze: D10. The frontend uses fake data until the real API is ready. Start connecting real APIs from D5.

Responsibilities

F1: Frontend Lead

- Build the login, register, repo list and add-repo screens
- Build the findings table with filters (severity, ecosystem, has-fix, direct/transitive)
- Build the refresh button, scan history and "new/resolved" badges
- Connect these screens to the real API from D5
- Own the frontend code structure and fake data setup

F2: Frontend, then Backend Helper

- **D1-D5:** Build the finding detail drawer (CVEs, score, fix version) and the SBOM tab
- **From D5:** Build the findings and components API endpoints with B1
- Build the fallback lookup (CVE ID to score) from cvelistV5
- Write tests and the README

B1: Backend Lead

- Set up the database, models and local `docker-compose` (Mongo, Redis)
- Build register/login with JWT, and make each user see only their own data
- Build the job queue and worker (clone, scan, save results)
- Build the repo, scan and refresh endpoints
- Validate inputs (only `github.com/owner/repo` URLs allowed)

B2: Scanner Engineer

- Clone repos safely (shallow clone, size and time limits, delete after scan)
- Read lockfiles (`package-lock.json`, then `poetry.lock`) into a full package list
- Ask OSV which packages are vulnerable
- Turn CVSS vectors into scores and severity levels
- Pick the right fixed version for each vulnerability
- Save the full package list (for the SBOM tab)

Everyone

- Agree on the API shape on D1 and freeze it by end of D2
- Test on real repos from D7
- Feature freeze on D10, then only bug fixes